

SW Engineering CSC648-848 Fall 2025

Project/application title and name: TBD

Team Number: 04

Team Lead: Iliana Gallegos/igallegos@sfsu.edu

Front-End Lead: Leigh Apotheker

Back-End Lead: Darien Sngoeun

GitHub Maintainer: Yuhang Wei

Back-End Dev: Jonathan Tsang

Back-End Dev: Roxana Del Toro

Front-End Dev: Megha Rai

Milestone 1

History Table

Date Submitted	Date Revised
Wednesday, October 15, 2025	

Table of Contents

SW Engineering CSC648-848 Fall 2025.....	0
Table of Contents.....	1
Executive Summary.....	2
Personae.....	3
High-level Use Cases.....	6
Data Glossary/Description.....	8
List High Level Functional Requirements.....	11
Non-Functional Requirements.....	13
Competitive Analysis.....	15
High-level System Architecture and Technologies Used.....	16
Use of GenAI Tools for Milestone 1.....	17
Team and Roles.....	19
Team Lead Checklist to be Completed by Team Lead.....	20

Executive Summary

SFSU Tutor is a tutoring platform for the university. It is designed to connect San Francisco State University students with qualified and verified tutors. The system addresses a key limitation of existing tutoring services, which do not authenticate campus affiliation or align with SFSU's course offerings. Currently, students struggle to find reliable tutors who understand course expectations and tutors face barriers reaching SFSU students. The application provides three main services: course-based search and filtering that allows students to locate tutors by course number, subject, or language preference, tutor profile management where tutors can post availability, hourly rate, and teaching materials subject to administrative approval, and administrative moderation that ensures content and accounts comply with SFSU standards.

All users must register with an @sfsu.edu email address to maintain campus exclusivity. Communication between students and tutors happens through a one-round in-site messaging system to preserve privacy and professionalism. The interface is designed for accessibility and responsiveness across desktop and mobile browsers, and prioritizes ease of use, data privacy, and trust.

The project integrates React 18 and Vite for the front end, Node.js 20 with Express 5 for the back end, and MySQL 8 hosted on AWS EC2 with NGINX and PM2 for deployment. The design follows Agile software-engineering practices.

Team 04 consists of SFSU students collaborating across front-end, back-end, and management roles. The objective of this milestone is to establish the foundation for an MVP that demonstrates a secure and user-centered tutoring system for SFSU's academic community.

Personae

1. Rebecca Zhao - Student - Tutor Seeker

General Characteristics: Undergraduate or graduate students at SFSU who need tutoring in specific courses. They may be full-time students juggling classes, work, and other responsibilities.

Goals: Find reliable, affordable tutors who understand SFSU course content. Needs an easy way to browse, compare, and contact tutors.

Skills: Average web user, comfortable with search/browse but expect simple, mobile-friendly UI. Uses an @sfsu.edu email address to register.

Pain Points: Current services don't focus on SFSU courses, making it hard to trust tutor quality. Scheduling conflicts and unclear pricing are frequent frustrations.

2. Alex Rivera - Tutor - Graduate Student / TA

General Characteristics: Graduate students or teaching assistants at SFSU who want to offer tutoring. They are academically strong and often balance teaching, research, or other jobs.

Goals: Advertise their expertise, list courses covered, set hourly rate and availability, and build credibility within the SFSU community.

Skills: Tech-comfortable, able to upload information and manage a tutor dashboard.

Pain Points: Difficult to reach SFSU students through general tutoring platforms. Struggle with last-minute cancellations and lack of institutional trust from generic sites. Wants visibility only to verified SFSU users.

3. Yuki Tanaka - Administrator - System Manager

General Characteristics: SFSU staff member or assigned system admin maintaining and moderating the tutoring platform.

Goals: Ensure safety of users and SFSU exclusivity. Approve or deny new tutor profiles, remove inappropriate users/content and resolve user reports to maintain a safe and reliable service.

Skills: Strong technical and policy knowledge, comfortable using an admin dashboard with review and reporting tools.

Pain Points: Large volume of submissions, repetitive approval tasks, and dealing with fake or incomplete tutor profiles. Needs efficient tools to reduce manual workload on reported items while maintaining fairness.

4. Michael Chen - International Student - ESL Learner

General Characteristics: International undergraduate or graduate students at SFSU whose first language is not English. They may be adjusting to U.S. academic culture and facing language barriers in courses.

Goals: Find tutors who understand the challenges of studying in a second language and can explain SFSU course material clearly. Prefer a way to filter or identify tutors who speak their native language, enabling clearer explanations when English becomes a barrier to understanding difficult concepts.

Skills: Comfortable with basic web navigation and mobile platforms. Uses language filter to locate bilingual tutors

Pain Points: Difficulty in finding tutors able to explain SFSU course material and expectations in a second language. Time zone confusion when scheduling (especially for evening classes after work) if international students are studying remotely from their home country.

High-level Use Cases

Use Case 1: Rebecca Zhao - Finding and Contacting a Tutor (Student Seeker)

An undergraduate SFSU student, Rebecca, new to campus and juggling classes while also working, is looking for help with challenging course topics. She has average web skills and uses her laptop or phone to access the platform. Rebecca logs in with her **@sfsu.edu** email address and opens the **Search Panel** on the tutoring site. She enters a **Course Number** or **Subject Tag** to find available **Tutor Profiles**. After reviewing the tutor's expertise, **Availability Summary**, and **Hourly Rate**, she picks a tutor who matches her needs. Rebecca then sends a single In-Site Message through the **Contact Form** telling the tutor what she needs help with. This message is saved through the system and is then displayed in the **Student Dashboard** area for later reference.

Use Case 2: Alex Rivera - Create and Manage a Tutor Profile (Tutor)

A graduate teaching assistant wants to earn extra income and help students in the computer science department. They are comfortable using web tools and familiar with uploading materials. The tutor signs up using their **@sfsu.edu** email address and opens the create **Tutor Profile**. They fill out their **Courses Covered**, **Subjects**, **Hourly Rate**, and **Availability Summary** then upload a **Tutor Profile Photo**. They also have the option to submit a **Sample Material** to showcase their work. Once their profile is submitted, it goes into **Admin Review**. After approval the **Tutor Profile** becomes visible in the **Search Results**. The Tutor can later log into their **Tutor Dashboard** to view **In-Site Messages** they received.

Use Case 3: Yuki Tanaka - Moderating Tutor Listings and User Reports (Administrator)

An SFSU staff member serving as the **Site Administrator**, who is experienced with online systems and campus policies. The admin logs into the **Admin Dashboard** to review **Pending Tutor Profiles** that are waiting for approval. They verify whether each profile has complete information and complies with content and policy rules. The admin **Approves** profiles and **Rejects** or **Removes** incomplete or inappropriate ones. They also monitor **Reported Items**(including messages), disable **User Accounts** if necessary, and **Admin Actions** are recorded.

Use Case 4: Michael Chen - Filter for Bilingual Tutors (International Student)

An **international SFSU student** from Japan who is studying business and taking difficult electives. English is their second language and they prefer educational support in Japanese. They are comfortable using websites but prefer simple navigation and mobile access. The student logs into the tutoring app with their **@sfsu.edu** email address and opens the **Search Panel**. They search by **Course Number** and apply a **Language Filter** for Japanese. The system displays **Tutor Profiles** that list **Languages Spoken** matching the filter. The student reads the **Tutor Profile**, checks the **Availability Summary** and sends a short **In-Site Message** requesting assistance in Japanese. This message is saved through the system and is then displayed in the **Dashboard** area for later reference.

Data Glossary/Description

Tutor-Related Items

1. **Tutor:** registered SFSU student providing tutoring services. They can create and update their Tutor Profile, manage profile status, and view Tutor Dashboard.
2. **Tutor Profile:** information created by a tutor containing Courses Covered, Subjects, Hourly Rate, and Availability Summary. After approval by an admin the profile becomes visible.
3. **Pending Tutor Profile:** Tutor Profile awaiting Admin Review. Not visible in searches until approved.
4. **Tutor Profile Photo:** image on a Tutor Profile. Visible after admin approval and is subject to policy
5. **Courses Covered:** the list of SFSU courses that a tutor can teach. Used as a primary search filter within the Search Panel and displayed on each Tutor Profile to help students find tutors with specific SFSU courses.
6. **Availability Summary:** the tutor's displayed time windows for offering tutor sessions (e.g., Mon-Wed 3-6pm).
7. **Hourly Rate:** tutor's displayed hourly cost for reference only. No in-app payments or transaction functionality.
8. **Languages Spoken:** list of languages a tutor can use to teach. Displayed on the Tutor Profile
9. **Sample Material:** optional teaching materials linked to a Tutor Profile. Visible after admin approval and is subject to policy

10. **Tutor Dashboard:** personal view for tutors showing received In-Site Messages and profile status.
11. **Subjects / Subject Tag:** topic or skill category that is linked to a tutor's expertise (e.g., Calculus, Programming). Used for filtering and browsing tutors.
12. **Contact Form:** form within a Tutor Profile page that is used by students to send one In-Site Message. Accessible only to logged-in SFSU users.

Student-Related Items

1. **Student:** registered SFSU user seeking tutoring help. They can search for a Tutor Profile, send one In-Site Message, and view the Student Dashboard.
2. **Student Dashboard:** personal view for students showing sent In-Site Messages.
3. **In-Site Message:** single message a student sends to a tutor through the Contact Form. This is stored in the system and is visible to both the sender and recipient. No chatting or threaded conversations are allowed.

Administrator-Related Items (Moderation)

1. **Administrator:** system manager responsible for reviewing and approving Tutor Profiles. They manage Reported Items by performing Admin Actions such as approve, reject, remove, or disable user accounts.
2. **Admin Dashboard:** admin control view for reviewing Pending Tutor Profiles, Reported Items, and logged Admin Actions.
3. **Admin Action:** recorded action made by an admin such as approve, reject, remove, or disable user.

4. **Reported Item:** content or user that's flagged by a student or tutor for inappropriate or inaccurate information. This is reviewed by an admin to be resolved.

Search & Discovery

1. **Search Panel:** feature where users enter Course Number, Subject Tag, and Language Filter criteria to find tutors.
2. **Search Results:** List of Tutor Profiles returned from the Search Panel query. Clicking on one of the results opens the full tutor profile page
3. **Course Number:** Official SFSU course code (e.g., CSC 317) used to classify and search tutor listings.
4. **Language Filter:** optional search filter used to narrow results by Languages Spoken

Account

1. **User Account:** entity associated with a student, tutor, or administrator. Defines permissions and dashboard access.

List High Level Functional Requirements

Unregistered Users

1. Unregistered Users shall be able to browse and search Tutor Profiles without logging in
2. Unregistered Users shall be able to filter search by Course Number, Subject Tag, and/or Language
3. Unregistered Users shall be able to open and view a Tutor Profile from Search Results

Registered Students

4. Registered Students shall be able to send one In-Site Message to a selected tutor through the Contact Form
5. Registered Students shall be able to view their sent In-Site Message in the Student Dashboard
6. Registered Students shall be limited to a single In-Site Message per tutor
7. Registered Students shall be able to submit a Reported Item on a profile

Registered Tutors

8. Registered Tutors shall be able to create a Tutor Profile which contains Courses Covered, Subjects, Hourly Rate, Languages Spoken, and Availability Summary
9. Registered Tutors shall have newly created or edited Tutor Profiles placed into Pending status for Admin Review
10. Registered Tutors shall have their Tutor Profiles visible in Search Results only after admin approval
11. Registered Tutors shall be able to edit their Tutor Profile and resubmit it for Admin Review
12. Registered Tutors shall be able to view received In-Site Messages in the Tutor Dashboard
13. Registered Tutors shall be able to view the current approval status (pending, approved, rejected) of their profile

Administrators

14. Administrators shall be able to review Pending Tutor Profiles and set approval status to approved or rejected

15. Administrators shall be able to remove an Approved Tutor Profile from public view
16. Administrators shall be able to review Reported Items and record an Admin Action (approve, reject, remove listing, disable user)
17. Administrators shall be able to disable or re-enable a User Account
18. Administrators shall be able to filter moderation lists by approval status (pending, approved, rejected, removed)
19. Administrators shall have each moderation decision recorded as an Admin Action with a timestamp and the affected entity

System

20. System shall require authentication with an @sfsu.edu email address before any non-public action
21. System shall restrict the Contact Form to authenticated SFSU users only
22. System shall persist all data items necessary to render Tutor Profiles, Search, and Dashboards
23. System shall prevent Tutors from replying to In-Site Messages to enforce one-round, non-chat communication
24. System shall record the submitter and timestamp for each Tutor Profile creation/edit and each In-Site Message

Non-Functional Requirements

1. Application shall be developed, tested and deployed using tools and servers approved by Class CTO and as agreed in M0.
2. Application shall be optimized for standard desktop/laptop browsers e.g. must render correctly on the two latest versions of two major browsers.
3. All or selected application functions shall be rendered well on mobile devices (no native app to be developed).
4. Posting of tutor information and messaging to tutors shall be limited only to SFSU students.
5. Critical data shall be stored in the database on the team's deployment server.
6. No more than 50 concurrent users shall be accessing the application at any time.
7. Privacy of users shall be protected.
8. The language used shall be English (no localization needed).
9. Application shall be very easy to use and intuitive.
10. Application shall follow established architecture patterns.
11. Application code and its repository shall be easy to inspect and maintain.
12. Google Analytics shall be used.
13. No e-mail clients shall be allowed. Interested users (clients) can only message service providers via in-site messaging. One round of messaging (from client to service provider) is enough for this application. No chat functions shall be developed or integrated.
14. Pay functionality (e.g. paying for goods and services) shall not be implemented nor simulated in UI.
15. Site security: basic best practices shall be applied (as covered in the class) for main data items.

16. Media formats shall be standard as used in the market today.
17. Modern SE processes and tools shall be used as specified in the class, including collaborative and continuous SW development and GenAI tools.
18. The application UI (WWW and mobile) shall prominently display the following exact text on all pages:
"SFSU Software Engineering Project CSC 648-848, Fall 2025. For Demonstration Only"
at the top of the WWW page Nav bar (Important so as to not confuse this with a real application).

Competitive Analysis

Key Functions	Wyzant	Tutor.com	City college(CCSF)	SFSU Tutor
Search & Filter Tutors	✓	✓	✓	✓+ (advanced filters)
In-App Messaging	✓	✓	✗ (email only)	✓+ (secure in-site message, SFSU only)
Mobile Responsive	✓	✓	✗ (desktop site)	✓+ (design for laptop and mobile)
University Access	✗	✗	✓ (CCSF students only)	✓+ (SFSU users with "@sfsu.edu" emails)
Admin Approval for Tutor listings	✗	✓ (limited)	✓ (staff-reviewed)	✓+ (mandatory admin)

Summary: We designed **SFSU Tutor** to solve the problems we actually run into as SFSU students. Big public sites like Wyzant and Tutor.com are fine for general matching, but they don't **verify campus affiliation** and moderation is limited. CCSF's portal is closer to a school model, but it's **desktop-only** and relies on **email back-and-forth**. Our version keeps things simple and campus-safe: we use **advanced filters** (subject, major) so students find the right tutor fast, keep all messages **inside the site** via secure messaging, and restrict access to **@sfsu.edu** accounts only. Every tutor post goes through **mandatory admin review** to keep quality high, and the UI is **fully responsive** on laptop and phone. Per class policy, we **don't include payments**; the focus is trust and usability. In short, **SFSU Tutor** combines the best parts of existing platforms and adds what we actually need on campus **verification, moderation, and a smoother mobile experience**.

High-level System Architecture and Technologies Used

Stack

- **Server Host:** Amazon AWS EC2 (t3.micro)
- **Operating System:** Ubuntu 24.04 LTS
- **Database:** MySQL 8.0.43
- **Web Server / Reverse Proxy:** NGINX 1.24.0
- **Runtime:** Node.js 20 LTS
- **Server-Side Language:** JavaScript (ECMAScript 2023)

Additional Technologies

- **Web Framework:** Express 5.1.0
- **Frontend:** React 18 + Vite 5, Tailwind CSS
- **Process Manager & Deploy:** PM2 5 (with pm2 deploy)
- **CI/CD:** GitHub Actions (auto-deploy on push to main)
 - We use Github Actions to automatically run pm2 deploy on every push to main. This gives us consistent repeatable deployments with an audit trail. SSH keys/host are stored securely as repo secrets.
- **SSL/TLS:** Let's Encrypt (Certbot)
 - Domain: <https://team04.duckdns.org>
- **Package Manager:** npm 10
- **IDE:** Visual Studio Code
- **Analytics:** Google Analytics 4

Browsers to Support

- Google Chrome
- Mozilla Firefox

Use of GenAI Tools for Milestone 1

Tools Used

We used ChatGPT (GPT-5, OpenAI, October 2025 version) as our primary GenAI tool.

Task	GenAI Tool	Usefulness	Description / Benefit
1. Writing the Executive Summary	ChatGPT	High	Helped brainstorm clear, concise marketing-style wording to explain our project idea to a non-technical audience. We used several drafts and edited manually to keep authentic content.
2. Persona Creation	ChatGPT	Medium	Generated sample persona templates and guided us in describing goals, frustrations, and user contexts. We customized all details based on our own interviews.
3. Use Case Storylines	ChatGPT	Medium	Used to check structure (“As a [user], I want to ... so that ...”) and to ensure each persona was linked to at least one use case.
5. High-level functional requirements	ChatGPT	Low	Helpful in ensuring there were no contradictions or redundancy in the requirements based on the use cases. Although I did use some given, it wasn’t really helping in generating the requirements because they would give the wrong format, too many unnecessary requirements, and sometimes the requirements they gave were non-functional ones.
7. Competitive Analysis Formatting	ChatGPT	Medium	Asked for example feature-comparison tables and clear phrasing of competitive advantages; actual feature data were collected by our team.

Other: Proofreading and Formatting Guidance	ChatGPT / Copilot	High	Suggested consistent section titles, numbering, and concise wording to match the Milestone 1 rubric.
--	----------------------	-------------	--

Examples of Prompts Used

- “Generate three sample personas for a web app connecting SFSU students with tutors.”
- “Rewrite this paragraph to sound professional but still concise for an executive summary.”
- “List functional requirements derived from these four use cases without using implementation language.”
- “In one table, compare Wyzant, Tutor.com, CCSF, and our SFSU Tutor platform by features only.”
- “Ensure that there are no contradictions in these high level functional requirements, comparing it to the given use cases and non-functional requirements”.

Overall Reflection

ChatGPT was most helpful for drafting and editing non-technical sections and for ensuring our document followed a clear professional structure. We learned that GenAI works best as a collaborative assistant for brainstorming and organization, not a replacement for team discussion or original content. All final writing was reviewed and edited by team members to ensure authenticity.

Team and Roles

Team Members	Email	Roles
Darien Sngoeun	dsngoeun@sfsu.edu	Back-End Lead
Leigh Apotheker	lapotheker@sfsu.edu	Front-End Lead
Yuhang Wei	ywei@sfsu.edu	GitHub Maintainer
Jonathan Tsang	jtsang1@sfsu.edu	Back-End Dev
Roxana Del Toro	rxdt@sfsu.edu	Back-End Dev
Megha Rai	mrai@sfsu.edu	Front-End Dev
Iliana Gallegos	igallegos@sfsu.edu	Team Lead

Team Lead Checklist to be Completed by Team Lead

☒ So far all team members are fully engaged and attending team sessions when required.

- **Done**

☒ The team found a time slot to meet outside of the class.

- **Done**

☒ Team ready and able to use the chosen back and front end frameworks and those who need to learn are w

- **Done**

☒ The team reviewed class slides on requirements and use cases before drafting Milestone 1.

- **Done**

☒ Team reviewed non - functional requirements from “How to start...” document and developed Milestone

- **Done**

☒ Team lead checked Milestone 1 document for quality, completeness, formatting and compliance with inst

- **Done**

☒ Team lead ensured that all team members read the final M1 and agree/understand it before submission.

- **Done**

☒ The team shared and discussed experience with GenAI tools among themselves.

- **Done**

☐ Github organized as discussed in class.

- **Done**